CS4243

AY26/27 Sem 1

github.com/ylchin

01. Introduction

- $f(x,y) = i(x,y)r(x,y)$ where f is intensity, i is illumination, and r is reflectance
- **Exposure time** - Time for incident light to reach sensor
- **Storage** - $b = MNK$ where img. has size $M \times N$ and K-bit depth
- Color to greyscale: $I = W_R R + W_G G + W_B B$ where $\sum_{i\in[R,G,B]} W_i = 1$
  - Common weights: $(.299, .587, .114)$
- **Norm. RGB** - $(r,g,b) = (\frac{R}{R+G+B}, \frac{G}{R+G+B}, \frac{B}{R+G+B})$
- $(R,G,B) \leftrightarrow (r,g,b) \leftrightarrow (r,g,I)$ where $I = \frac{R+G+B}{3}$
- **HSV Color Space**
  - Hue: Pure color (0 to 360)
  - Saturation: Mix pure color (1) with white light (0)
  - Value: Mix from black (0) to white (255)

02. Filtering

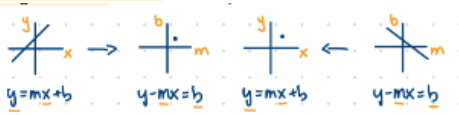
- **Point processing** - $x_{ij} = f(p_{ij})$
  - **Brightness** - $x_{ij} = p_{ij} + b$ (Clipping behavior)
  - **Intensity scaling** - $x_{ij} = ap_{ij}$ (Increased/decreased contrast)
  - **Normalization (Whitening)** - $x_{ij} = \frac{p_{ij}-\mu}{\sigma}$ $\sigma^2 = \frac{\sum_{i=0}^{I}\sum_{j=0}^{J}(p_{ij}-\mu)^2}{IJ}$
  - **Gamma** - $x_{ij} = 255(\frac{p_{ij}}{255})^\gamma$ where $\gamma > 0$
  - **Intensity Histogram** - No data on location
    - **Stretching** - $x = (p - f_{\min})(\frac{255}{f_{\max}-f_{\min}})$
    - **Equalization** - Turn cumulative distribution linear
      1. Get histogram: $h_k = \sum_{i=0}^{I}\sum_{j=0}^{J}\delta$ where $\delta = 1$ if $p_{ij} - k = 0$ and 0 otherwise
      2. Estimate CDF: $c_k = \frac{\sum_{l=1}^{k} h_l}{IJ}$
      3. Map $p_{ij} = k$ to new bin x_{ij}: $x_{ij} = 255\text{CDF}(k)$
         - Pros: More contrast than normalization, less prone to outliers than stretching
         - Cons: More expensive
    - Good for foreground and background separation: Bimodal histogram can be thresholded
- **Correlation Filtering** - Window (Moving average)
  - Motivation: Reduce noise
    - Impulse noise: Random white occurrences of pixels
    - Salt and pepper noise: Rand. white and black pixels
    - Gaussian noise: Variations in intensity from Gaussian distribution $p_{ij} = \hat{p}_{ij} + \eta$ where $\eta \sim N(\mu_n, \sigma_n)$
  - $x_{ij} = \sum_{u=-k}^{k}\sum_{v=-k}^{k} f_{uv} \cdot p_{i+u,j+v}$ where f is a **kernel** of weights
  - Notation: $X = P \otimes F$
  - **Box Filter** - Blurs image
  - Given kernel of width $2k+1$, add padding: $\lfloor \frac{2k+1}{2} \rfloor$
    - Filling methods: Zero-padding, wrap around, copy edge, reflect
  - **Gaussian Filter** - Nearest neighboring pixels have more weight
    - $f_{uv} = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$
    - Size of kernel: At some threshold, kernel will have lots of 0s, which is not useful
    - Variance: Determines smoothing effect
  - **Template Matching** - Set template as kernel and match occurs at local max.
    - **Normalized Cross-Correlation** - Normalizes using filter and input window
      - Motivation: Non-normalized is dominated by original image pixels
    - $x_{ij} = \frac{1}{|F||w_{ij}|}\sum_{u=-k}^{k}\sum_{v=-k}^{k} f_{uv} \cdot p_{i+u,j+v}$ where $|\cdot|$ is magnitude of kernel (i.e. Square root of sum of squares)
  - **Convolution Filtering** - Flip kernel in both directions, then do cross-correlation
    - $x_{ij} = \sum_{u=-k}^{k}\sum_{v=-k}^{k} f_{uv} \cdot p_{i-u,j-v}$
    - Notation: $X = F * P$
    - Convolution has nicer properties than correlation
    - Properties: Commutative, Associative, Distributive over addition, Scalar factor, Identity
  - **Non-Linear Filters** - Filters that perform non-linear operations (e.g. Median, Min., Max.)
    - Correlation and convolution are both linear operations
    - **Median Filtering**
      - Removes spikes: Good for removing impulse and salt and pepper noise
      - No new pixel values introduced
      - Edge preserving

03. Gradients

- **Gradient** - Vector at pixel pointing at most rapid change in intensity
  - $\nabla f = [\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}]$
  - In images, der. is finite: $f'(x) = \frac{f(x+1)-f(x-1)}{2}$
  - **Gradient direction** - orientation of edge normal: $\theta = \tan^{-1}(\frac{\partial f}{\partial x}/\frac{\partial f}{\partial y})$
  - **Edge strength** - gradient magnitude $||\nabla f|| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$
- **Sobel Filter**
  - Horizontal sobel filter (S_x): Detects vertical lines
  - Vertical sobel filter (S_y): Detects horizontal lines

$$S_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} \qquad S_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

  - Differentiation is sensitive to noise: Need to blur first
    - Trade-off: Blur removes noise, but blurs the edge
    - **Derivative of Gaussian** - Blur + Derivative merged into 1 filter; $\frac{\partial}{\partial x}(h * f) = (\frac{\partial}{\partial x}h) * f$
- **Laplace Filter** - 2nd Derivative Filter
  - $f''(x) = \lim_{h\to 0}\frac{f(x+h)-2f(x)+f(x-h)}{h^2}$
  - **Laplacian of Gaussian (LoG) Filter** - Combine Laplace filter with Gaussian
    - Produces zero-crossings at edge
- **Canny Edge Detector**
  1. Filter image with derivative of Gaussian
  2. Find magnitude and orientation of gradient
  3. Thin wide ridges down to single pixel width via non-maximum suppression
  4. Perform thresholding (hysteresis) and linking
- **Non-maximum Suppression** - For each pixel, check if it's local maximum along gradient direction. If yes, then keep. Else, 0.
- **Hysteresis thresholding** - Use a high threshold to start edge curves and low threshold to continue growing them
  - Motivation: Single high threshold can miss out some parts of the edge
- **Hough Transform**
  - Goal: Which points belong to what line?
  - $y = mx + b$ $\quad \frac{x}{a} + \frac{y}{b} = 1$
  - $x\cos\theta + y\sin\theta = \rho$
  - **Line fitting** - $E = \frac{1}{N}\sum_i (y_i - (mx_i + b))^2$
    - Goal: Find parameters m and b that min. E
    - Using gradient descent: $b = \bar{y} - m\bar{x}$ $\quad m = \frac{\sum_i (x_i-\bar{x})(y_i-\bar{y})}{\sum_i (x_i-\bar{x})^2}$ where $\bar{y} = \frac{\sum_i y_i}{N}$ and $\bar{x} = \frac{\sum_i x_i}{N}$
    - Cons: Choice of error function, outliers
  - **Hough Transform** - Parameterize shape and vote



    1. Initialize accumulator array $A(\theta, \rho) = 0$
    2. For each image edge point (x_i, y_i):
    (1) For each θ:
       i. Solve for $\rho = x_i\cos\theta + y_i\sin\theta$
       ii. $A(\theta, \rho) = A(\theta, \rho) + 1$
    3. Find maximum in $A(\theta, \rho)$
    4. Detected lines are given by $\rho^* = x\cos\theta^* + y\sin\theta^*$

04. Filter Banks, Texture and Learned Filters

- **Filter bank** - Collection of kernels detecting different local patterns (edges, bars, spots) at different orientations and scales.
  - Filter response $r_k(x,y) = (I * h_k)(x,y)$
  - K filters → K-dimensional feature vector at each pixel: $\mathbf{r}(x,y) = [r_1(x,y), \cdots, r_K(x,y)]^T$
- **Gabor Filters**
  - Sinusoid × Gaussian envelope; detects oriented patterns at specific frequencies/scales.
  - **1D Gabor** - $g(x) = e^{-\frac{x^2}{2\sigma^2}}\sin(2\pi fx)$
  - **2D Gabor** - $g(m,n) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{m'^2+\gamma^2 n'^2}{2\sigma^2}\right)\sin\left(\frac{2\pi m'}{\lambda} + \phi\right)$ where $m' = m\cos\omega + n\sin\omega$ and $n' = -m\sin\omega + n\cos\omega$
    - ω: Orientation of sinusoid.
    - λ: Wavelength; $f = 1/\lambda$.
    - ϕ: Phase shift.
    - γ: Aspect ratio ($\gamma = 1$: circular; otherwise elliptical).
    - σ: Gaussian envelope scale/decay.
  - Key points:
    - Gaussian envelope selects local spatial region.
    - Sinusoid selects orientation and frequency.
    - Higher frequency → narrower stripes; lower frequency have broader, smoother responses to coarser patters; higher-frequency filters with smaller, more localized responses.
    - Larger $\sigma \to$ larger spatial coverage.
    - Odd (sine) → edges; even (cosine) → bars/ridges.
    - Frequency controls pattern spacing; scale controls neighborhood size.
    - Usually $\lambda \propto \sigma$ to maintain a constant number of visible cycles.
- **Multi-scale reasoning**
  - Why? Structures have different characteristic sizes and apparent sizes.
  - **Gaussian pyramid** - Progressively blurred + downsampled images
    - **Downsampling** - Removing rows and column of pixels at a fixed stride
    - Gaussian smoothing suppresses fine details; prevents aliasing (false patterns introduced by downsampling) before downsampling
    - Same filter at higher pyramid levels covers larger regions of the original image
  - **Scale selection**
    - Measure responses across scales; choose local maximum where filter scale matches structure size
    - **LoG** - Second derivative of Gaussian; blob detector.
      $\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$
      - Scale normalised response $R(x,y;\sigma) = \sigma^2|\nabla^2(G_\sigma * I)(x,y)|$
      - σ^2: allows comparison across scales
      - **Difference of Gaussians (DoG)** - Cheaper LoG approximation
  - **Multi-scale representations** - Retain and concatenate responses from different scales instead of selecting only the maximum. $\mathbf{r} = [r_1^{\sigma_1}, \ldots, r_K^{\sigma_1}, r_1^{\sigma_2}, \ldots, r_K^{\sigma_2}]$
- **Textures** - Regional patterns of repeating elements; useful for describing material properties
  - Pipeline: filter bank → response maps → regional pooling → texture vector
  - Pixel-wise response are too localised → pool responses over region Ω.

$$t_k(\Omega) = \frac{1}{|\Omega|}\sum_{(x,y)\in\Omega}|r_k(x,y)|$$

$$\mathbf{t}(\Omega) = [t_1, \cdots, t_K]^T$$

  - Pooling summarises regional patterns, reducing sensitivity to exact positions
  - **Texture similarity** - $D(\Omega_a, \Omega_b) = ||\mathbf{t}(\Omega_a) - \mathbf{t}(\Omega_n)||_2$
  - Small distance indicates similar texture
  - Material Classification
    - Nearest neighbour: Assign class of closest training texture vector

$$i^* = \arg\min_i ||\mathbf{t}(\Omega) - \mathbf{t}(\Omega_i)||_2$$

    - Nearest class mean: Assign class of closest representative class mean

$$\mu_c = \frac{1}{N_c}\sum_{i:y_i=c}\mathbf{t}(\Omega_i)$$

$$\hat{c}(\Omega) = \arg\min_i ||\mathbf{t}(\Omega) - \mu_c||$$

- **Learned filters**
  - **Traditional CV** - Filtering → Pooling → Classifier; kernels manually specified
  - **CNN** - Learns kernels from data to minimise prediction error; convolutional layer = learned filter bank.
    - Convolving a filter on a single input gives a single feature map
    - K filters gives K output channels
    - Multiple layers learn increasingly complex features
  - **ReLU** - $ReLU(r) = \max(0, r)$
    - Introduces nonlinearity. Without it, stacked linear convolutions collapse into one effective kernel.
  - **Max Pooling** - Retains maximum activation within a local window; reduces resolution and sensitivity to small distortions.
  - **CNN Architecture** - Conv → ReLU → Max Pool
    - Repeated blocks → fully connected classifiers → class scores.
    - **Receptive field** - Original image region influencing an output activation.

05. CNNs

- **Image classification**
  - Goal: Assign a discrete label from K classes to an entire image

$$\mathbf{x} \in \mathbb{R}^{H \times W \times C}$$

$$f_\theta(\mathbf{x}) = \mathbf{z} \in \mathbb{R}^K, \hat{y} = \arg\max_k z_k$$

  - Accuracy: $\frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\hat{y}_i = y_i]$
  - **Classifier stages** - $\mathbf{x} \to^{\phi_{\theta_f}} \mathbf{h} \to^{g_{\theta_c}} \mathbf{z} \to^{\arg\max} \hat{y}$
    - ϕ_{θ_f}: Feature extractor; maps pixels to representation $\mathbf{h}$.
    - g_{θ_c}: Classifier; maps features to K class scores.
    - Traditional CV: hand-designed feature extractor, learned classifier.
    - Deep learning: both learned jointly (end-to-end).
  - Data pipeline: labeled training data → train classifier → tune hyperparameters on validation set → evaluate on unseen test set.
- **Neural Networks & MLPs**
  - **Perceptron** - Weighted sum + bias + nonlinear activation $a = \sigma(\mathbf{w}^T\mathbf{x} + b)$
  - **Multi-layer perceptron (MLP)** - Connected perceptrons across multiple layers.

$$\mathbf{a}^{(0)} = \mathbf{u}, \mathbf{a}^{(l)} = \sigma(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$

  - Output layer (classification head): $\mathbf{z} = W^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}$
  - Fully connected layer parameter count: $\#\text{params} = n_{\text{in}}n_{\text{out}} + n_{\text{out}}$
  - MLP limitations for images:
    - Requires flattening $H \times W \times C \to HWC$
    - Ignores explicit spatial neighborhood relationships.
    - Every hidden unit connects to every input pixel →, resulting in a large parameter count
  - MLPs are commonly used as CNN classification heads
- **Convolutional Neural Networks**
  - Why convolution?
    - Local connectivity: Each neuron observes a local neighbourhood (receptive field)
    - Parameter sharing: Same kernel applied at different spatial locations
    - Translation equivariance: Translating input produces a corresponding translation in feature matps
    - Substantially fewer parameters than fully connected layers
  - **Strided convolution** - Downsamples feature map; reduces resolution and computation
  - **Dilated convolution** - Enlarges receptive field without extra parameters or reducing spatial resolution
  - Output dimension: $H_{\text{out}} = \lfloor \frac{H+2P-D(K-1)-1}{S} \rfloor + 1$
  - For no dilation ($D = 1$): $H_{\text{out}} = \lfloor \frac{H+2P-K}{S} \rfloor + 1$
  - H: input size; P: padding; K: kernel; S: stride; D: dilation
  - **CNN Feature Progression**
    - Typical architecture: Input → [Conv → Activation → Pooling]^n → Classification Head
    - As network depth increases:
      - Spatial resolution ↓
      - Number of channels ↑
      - Receptive field ↑
      - Features become increasingly complex and task-specific.
    - **Backbone** - Extracts reusable feature maps
    - **Task-specific head** - Converts features into class logits, bounding boxes, segmentation masks
  - **Classification Head** - Turns features into a prediction by selecting the class with the largest logit, or class score.
    - Logits: $\mathbf{z} = W^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}$
    - Softmax: $p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$, $\sum_k p_k = 1$, $\arg\max_k z_k$
    - Logits are unrestricted scores; softmax converts them into class probabilities.
    - **Cross-entropy** - Continuous loss penalising low probability assigned to the correct class $L_{CE} = -\log p_y$
    - 0–1 classification loss gives no useful smooth learning signal; cross-entropy does.
- **Object detection**
  - Task: detect the location of all object instances of all classes
  - Each object prediction: $(c_i, s_i, x_i, y_i, w_i, h_i)$
  - c_i: Class; s_i: confidence.(x_i, y_i): Top-left box coordinates.w_i, h_i: Width and height.
  - Classification: What is in the image?
  - Detection: What is where?
  - **Window-based detection**
    1. Train object/non-object classifier.
    2. Generate candidate windows at multiple locations and scales.
    3. Score each candidate independently.
    4. Resolve detections using thresholding and NMS
    - Challenges: illumination, pose, occlusion, clutter, viewpoint and intra-class variation. Background class is highly diverse.
    - **Non-Maximum Suppression (NMS)** - Retain highest-scoring detection; suppress lower-scoring boxes with excessive overlap.
  - **Detection Evaluation**
    - **Intersection over Union** - $IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}$
    - $Precision = \frac{TP}{TP+FP}$, $Recall = \frac{TP}{TP+FN}$
    - **Average Precision (AP)** - Area under precision-recall curve for one class at a fixed IoU threshold $mAP = \frac{1}{C}\sum_{c=1}^{C} AP_c$
    - Some protocols also average across IoU thresholds 0.50–0.95.
    - Low precision → many FP; low recall → many FN.
    - TP: matched detection with IoU $\geq t$ (usually $t = 0.5$); FP: unmatched/low-IoU detection; FN: missed GT object; TN not meaningful.
- **CNN-based object detection**
  - **R-CNN** - Generate proposals; run CNN separately on each region; classify + regress boxes
  - **Fast R-CNN** - Extract whole-image CNN features once; RoI pooling; external proposals
  - **Faster R-CNN** - Integrates learned Region Proposal Network (RPN)
  - R-CNN is slow because approximately 2000 proposed regions each pass through the CNN.
  - Two-stage detection: Region proposals → classify and refine bounding boxes.
  - One-stage detection: Predict classes and bounding boxes directly and densely.
  - **YOLO** - You Only Look Once
    - One-stage detector. Partition image into grid cells with predefined anchor boxes in the version shown in the slides.
    - Predict per location/anchor:
      - Objectness: $P(\text{object})$
      - Class score: $P(\text{category} \mid \text{object})$
      - Bounding-box coordinate adjustments.
    - Loss: $\mathcal{L} = \mathcal{L}_{\text{class}} + \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{obj}}\mathcal{L}_{\text{objectness}}$
  - **Modern YOLOs**
    - Multi-scale feature fusion → handles different object sizes. Decoupled class and box prediction branches.

06. Segments

- Goal: Separate image into coherent regions
- Idea: **Clustering** - Group similar data points together
- Challenges: What makes 2 points same/different? Choice of features (e.g. Color, Intensity, Position), Which clustering algorithm?
- **Segmentation Types**
  - Semantic: class per pixel; same-class objects share label
  - Instance: separates individual object instances
  - Panoptic: semantic + instance segmentation
  - Superpixel: compact similar-pixel region; not necessarily an object
- **k-Means Clustering** - Iteratively re-assign points to nearest cluster center
- k-Means: $y_i = \arg\min_j ||p_i - c_j||^2$, $c_j = \frac{\sum_{i:y_i=j} p_i}{\sum_{i:y_i=j} 1}$
  1. Randomly initialize the cluster centers $c_1, \ldots, c_K$
  2. For each point p_i, find the closest c_j to put p_i in
  3. Set c_j to be mean of points in cluster j
  4. Repeat, if c_j have changed up to some threshold
  - Pros: Simple, Converges to local min.
  - Cons: Setting K, Sensitive to initial centers (Since k-means converges to local min.), Sensitive to outliers (Can add more clusters), Assumes spherical clusters
- **Simple Linear Iterative Clustering (SLIC) Superpixels**
  - **Superpixel** - Group of pixels that share common traits
    - Application: Inputs to other CV algo. since more compact representation with perceptual meaning
  - Num. of pixels: n_{tp}; Target num. of superpixels: n_{sp}
  - Initial width of each superpixel: $s = \sqrt{n_{tp}/n_{sp}}$
  - Features: $z = [r, g, b, x, y]$
  - Color distance: $d_c = ||\langle r_j, g_j, b_j\rangle - \langle r_i, g_i, b_i\rangle||$
  - Spatial distance: $d_s = ||\langle x_j, y_j\rangle - \langle x_i, y_i\rangle||$
  - Scaling factors: d_{cm} and $d_{sm} = s$ set as max. expected values of d_c and d_s respectively
  - $D = \sqrt{(\frac{d_c}{d_{cm}})^2 + (\frac{d_s}{d_{sm}})^2} = \sqrt{d_c^2 + (\frac{d_s}{s})^2 c^2}$
  1. Split img. into grid of size $s \times s$. Set cluster centers as lowest gradient position in 3×3 neighborhood from superpixel center to speed up convergence since initialize on value common to surrounding.
  2. For each cluster center, check distance to all pixels within $2s \times 2s$ neighborhood. Assign pixels to closest checked center.
  3. Update cluster centers using mean and repeat if not converged (Same as k-Means)
  4. Optional: Replace superpixel region by average value to create stained glass effect
  - Modification of k-Means: Not random initialization, Compute pixel's distance only to closest set of cluster centers
- **Neural Network Segmentation**
  - **Dense classification** - Classify every pixel: $H \times W \times 3 \to H \times W \times K \to H \times W$
  - **Encoder** - Extract semantic features + downsample **Decoder** - Upsample to recover spatial resolution
  - Upsampling: nearest-neighbour (blocky), bilinear (smooth), transposed conv. (learned; insert zeros then convolve)
  - **FCN**
    - FCN-32: no skips; FCN-16: +pool4; FCN-8: +pool3
    - Skip connections combine semantic + high-resolution features
  - **U-Net** - Encoder-decoder with skip concatenations; preserves boundaries
    - Final conv: K channels; softmax across channels; CE per pixel
  - **Evaluation**
    - Pixel accuracy poor for small foreground
    - $IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}$, $Dice(A, B) = \frac{2|A \cap B|}{|A| + |B|}$
    - CE: pixel labels; overlap loss: region overlap
    - Dice and IoU rank predictions identically; Dice gives larger numerical scores.
  - **Mask R-CNN** - Detection + mask head → class, box, mask per instance
- **Promptable Segmentation**
  - **Prompt** - Specifies target at inference without retraining; points, box, mask
  - **SAM** - Image encoder + prompt encoder → mask decoder → mask
    - Positive/negative points include/exclude regions; prompts may be ambiguous
  - **SAM 2** - Video segmentation + memory **SAM 3** - Concept/text or exemplar prompts